

API Test Report — AI Platform Integration Testing

Overview

This report documents API testing conducted against a representative AI chat completion API: Hugging Face's Inference Router, using Llama 3.1 8B as the underlying model. The testing approach mirrors what a Solution Consultant would run before certifying a client integration as ready for go-live with an AI platform like Travtus Adam.

The purpose was to validate API behaviour across authentication, error handling, data validation, and boundary conditions. These checks are designed to catch the failure modes most likely to surface during a live client deployment.

API tested: Hugging Face Inference Router (chat completions and model metadata endpoints)

Tool used: Postman

Total requests built: 8

Date: [today's date]

Test Results

#	Test Name	Expected Status	Actual Status	Pass/Fail
1	Valid chat message	200	200	Pass
2	Empty message body	200 or 400	200	Pass
3	Invalid API key	401	401	Pass
4	No API key	401 or 403	401	Pass
5	Long message boundary	200	200 (3.43s)	Pass
6	Wrong data type	400 or 422	400	Pass
7	Model info (valid key)	200	200	Pass

#	Test Name	Expected Status	Actual Status	Pass/Fail
8	Invalid model endpoint	400 or 404	400	Pass

All 8 requests passed their assertions. 16 individual test cases ran across the suite, all green.

Key Findings

Authentication testing. The API consistently returns 401 for both missing and invalid keys, which is the correct behaviour. If a client integration loses its API key mid-session, it will fail loudly with a clear error rather than silently passing bad data downstream. For a Solution Consultant onboarding a new client, this means the auth layer should surface key rotation issues immediately rather than letting them quietly corrupt resident communications.

Error handling. The data type validation test returned a clear, structured error explaining exactly what was wrong: "body unmarshal json: cannot unmarshal number into Go struct field." Client-side code can parse this and present it in a useful way to property managers. The empty message test returned 200 with the AI generating a generic response, which means an integration needs to apply its own input validation rather than relying on the API to reject empty inputs.

Boundary behaviour. The long message test (roughly 850 characters of resident input) completed successfully in 3.43 seconds. This is well within an acceptable response time for resident communications. For Travtus Adam, this suggests that even verbose maintenance complaints or detailed lease queries can be processed without timeout risk. However, a three-second response window means the client UI should show a "thinking" indicator so users do not assume the system has stalled.

Pre-Deployment Recommendations

Before any client goes live with an AI communication platform like Adam, the following API checks should be run as part of the standard onboarding validation:

1. Confirm API keys are stored in environment variables, not hardcoded in any client config file or version control.

2. Set up monitoring alerts for any spike in 401 responses, which signal key expiry or misconfiguration.
3. Test the exact API call patterns the client's volume will generate, including burst conditions during high-traffic periods like rent due dates or building emergencies.
4. Validate error response formats so client-side code can handle failures gracefully and surface meaningful messages to property managers.
5. Document expected response times under realistic message-length distributions, so escalation rules can be calibrated to actual performance rather than assumed performance.
6. Test with deliberately malformed inputs (wrong data types, oversized payloads, special characters) to confirm the integration layer's input validation catches issues before they reach the AI.

The tests in this suite represent a baseline for any AI platform deployment. Specific client integrations would extend this with tests against the client's own systems (Yardi, Zendesk, Salesforce, SendGrid) using the same methodology.

1. Valid chat message 200 200 Pass

The screenshot displays the Postman interface for a workspace named 'Oluwatobi Ayanreti's Workspace'. The active collection is 'AI Platform API Test Suite', and the selected test is 'POST - Valid chat message'. The request is a POST to 'https://router.huggingface.co/v1/chat/completions' with the following headers:

Key	Value	Description
Content-Type	application/json	
Authorization	Bearer {{hf_token}}	

The response is a 200 OK status, received in 2.82 seconds with a 1.74 KB body. The response body is a JSON object:

```

{
  "id": "9714e1412a49047bd126de6654ee52ff",
  "object": "chat.completion",
  "created": 1778107366,
  "model": "meta-llama/llama-3.1-8b-instruct",
  "choices": [
    {
      "text": "..."
    }
  ],
  "usage": {
    "prompt_tokens": 1,
    "completion_tokens": 1,
    "total_tokens": 2
  },
  "system_fingerprint": "..."
}

```

The interface also shows a sidebar with other tests in the collection, including 'Empty message body', 'Invalid API key', 'No API Key', 'Boundary test (long message)', 'Wrong data type (number instead of string)', 'Model info (valid key)', and 'Invalid model endpoint'. The bottom status bar indicates 'Waiting for approval...'.

2. Empty message body 200 or 400 200 Pass

The screenshot shows the Postman interface with a workspace named "Oluwatobi Ayanreti's Workspace". The active collection is "AI Platform API Test Suite". The selected test is "POST - Empty message body". The request is a POST to "https://router.huggingface.co/v1/chat/completions". The test script is as follows:

```
1 pm.test("Empty message handled without server crash", function() {
2   pm.expect(pm.response.code).to.be.oneOf([200, 400, 422]);
3 });
4 pm.test("Response body is not empty", function() {
5   pm.expect(pm.response.text()).to.not.be.empty;
6 });
```

The response is a 200 OK status with a response time of 897 ms and a body size of 1.25 KB. The response body is a JSON object:

```
{
  "id": "c9074fb3c0a1142a4ffc4adda7840a0",
  "object": "chat.completion",
  "created": 1778108486,
  "model": "meta-llama/llama-3.1-8b-instruct",
  "choices": [
    {
      "text": ""
    }
  ],
  "usage": {
    "prompt_tokens": 5,
    "completion_tokens": 0,
    "total_tokens": 5
  }
}
```

The right sidebar shows the "Adding visualization script" section with a "Waiting for approval..." message.

3. Invalid API key 401 401 Pass

The screenshot shows the Postman interface with the same workspace and collection. The selected test is "POST - Invalid API key". The request is a POST to "https://router.huggingface.co/v1/chat/completions". The test script is as follows:

```
1 pm.test("Returns 401 Unauthorized", function() {
2   pm.response.to.have.status(401);
3 });
4 pm.test("Error message present in response", function() {
5   pm.expect(pm.response.text()).to.not.be.empty;
6 });
```

The response is a 401 Unauthorized status with a response time of 732 ms and a body size of 975 B. The response body is a JSON object:

```
{
  "error": "Invalid username or password."
}
```

The right sidebar shows the "Adding visualization script" section with a "Waiting for approval..." message.

4. No API key 401 or 403 401 Pass

The screenshot shows the Postman interface with a workspace named "Oluwatobi Ayanret's Workspace". The selected collection is "AI Platform API Test Suite". The current request is a POST to "https://router.huggingface.co/v1/chat/completions". The test script is as follows:

```
1 pm.test("Returns auth error without API key", function() {  
2   pm.expect(pm.response.code).to.be.oneOf([401, 403]);  
3 });
```

The response is a 401 Unauthorized status with a message: "Unauthorized access. Please check your credentials or authorization". The response body is a large white image with a yellow emoji and the text "401 Unauthorized access. Please check your credentials or authorization".

5. Long message boundary 200 200 (3.43s) Pass

The screenshot shows the Postman interface with the same workspace and collection. The current request is a POST to "https://router.huggingface.co/v1/chat/completions". The test script is as follows:

```
1 pm.test("Long message handled successfully", function() {  
2   pm.response.to.have.status(200);  
3 });  
4 pm.test("Response time acceptable under load", function() {  
5   pm.expect(pm.response.responseTime).to.be.below(45000);  
6 });  
7 pm.test("Response contains AI-generated reply", function() {  
8   const json = pm.response.json();  
9   pm.expect(json).to.have.property('choices');  
10  pm.expect(json.choices[0].message).to.have.property('content');  
11 });
```

The response is a 200 OK status with a response time of 3.43 s. The response body is a JSON object:

```
{  
  "id": "e0af98399e80eeffe8e4da034318c52af",  
  "object": "chat.completion",  
  "created": 1778109357,  
  "model": "meta-llama/llama-3.1-8b-instruct",  
  "choices": [  
    {  
      "index": 0,  
      "message": {  
        "role": "assistant",  
        "content": "I'm a large language model trained by OpenAI, based on the GPT-4 architecture. I can help with a wide range of tasks, from creative writing to answering questions. I don't have a physical form or consciousness, but I can generate text based on the input I receive. How can I assist you today?"  
      },  
      "logprobs": null,  
      "finish_reason": "stop",  
      "usage": {  
        "prompt_tokens": 15,  
        "completion_tokens": 100,  
        "total_tokens": 115  
      }  
    }  
  ],  
  "usage": {  
    "prompt_tokens": 15,  
    "completion_tokens": 100,  
    "total_tokens": 115  
  }  
}
```

6. Wrong data type 400 or 422 400 Pass

The screenshot shows the AI Platform API Test Suite interface. The left sidebar lists collections, including 'AI Platform API Test Suite' and 'My Collection'. The main panel displays a POST request to 'https://router.huggingface.co/v1/chat/completions'. The 'Post-response' tab shows a 400 Bad Request status. The response body is a JSON object with the following structure:

```
{
  "code": 400,
  "reason": "CODEC",
  "message": "body unmarshal json: cannot unmarshal number into Go struct field ChatCompletionRequestAlias.messages.content of type []openai.ChatMessagePart",
  "metadata": {}
}
```

The right sidebar shows the 'Adding visualization script' section with a 'Save HTTP request' button and a 'Waiting for approval...' message.

7. Model info (valid key) 200 200 Pass

The screenshot shows the AI Platform API Test Suite interface. The left sidebar lists collections, including 'AI Platform API Test Suite' and 'My Collection'. The main panel displays a GET request to 'https://huggingface.co/api/models/meta-llama/Llama-3.1-8B-Instruct'. The 'Post-response' tab shows a 200 OK status. The response body is a JSON object with the following structure:

```
{
  "_id": "6698d8a0653e4babe21e1e7d",
  "id": "meta-llama/Llama-3.1-8B-Instruct",
  "private": false,
  "pipeline_tag": "text-generation",
  "library_name": "transformers",
  "tags": [
    "25"
  ],
  "downloads": 9683014,
  "likes": 5794,
  "modelId": "meta-llama/Llama-3.1-8B-Instruct",
  "author": "meta-llama",
  "sha": "0e9e39f249a16976918f6564b8830bc894c89659",
  "lastModified": "2024-09-25T17:00:57.000Z",
  "gated": "manual",
  "disabled": false,
  "widgetData": [
    "4"
  ]
}
```

The right sidebar shows the 'Adding visualization script' section with a 'Save HTTP request' button and a 'Waiting for approval...' message.

8. Invalid model endpoint 400 or 404 400 Pass

The screenshot displays the AI Platform API Test Suite interface. The left sidebar shows a collection of tests under 'AI Platform API Test Suite', with 'POST - Invalid model endpoint' selected. The main panel shows the details of this test, which is a POST request to 'https://router.huggingface.co/v1/chat/completions'. The request headers are 'Content-Type: application/json' and 'Authorization: Bearer {{hf_token}}'. The response is a '400 Bad Request' with a status of 400, a response time of 409 ms, and a body of 976 B. The response body is shown as a JSON object: {'error': '400 Bad Request'}. The right sidebar shows the 'Adding visualization script...' section.

POST - Invalid model endpoint

POST https://router.huggingface.co/v1/chat/completions

Headers

Key	Value	Description
Content-Type	application/json	
Authorization	Bearer {{hf_token}}	

Body

400 Bad Request - 409 ms - 976 B

error {4}